

# 24CSA212 – Full Stack Frameworks

---

DATA MODELLING WITH MONGODB

# Data Modelling in MongoDB

- MongoDB data modeling focuses on **structuring documents and collections in a way that matches application access patterns, improves query performance, and supports future scalability using MongoDB's flexible schema.**
- Designs document and collection structures for **efficient storage and queries.**
- Leverages **flexible schema** for dynamic and evolving data.
- **Optimizes** for application **access patterns** (reads/writes).
- **Improves database performance** via efficient storage and indexing.
- Plans for future scalability and changing requirements.



**Embed vs Normalized**  
Choose Wisely



**Schema Design**  
Optimize for Queries



**Use Indexes**  
Speed Up Searches



**Relationship Types**  
Model Connections

# MongoDB Data Model Designs

- For modeling data in MongoDB, two strategies are available. These strategies are different and it is recommended to analyze scenario for a better flow.
- The two methods for data model design in MongoDB are:
  - **1. Embedded Data Model**
  - **2. Normalized Data Model**

# Embedded Data Model

- The Embedded Data Model stores related data within a single MongoDB document, improving read performance by keeping frequently accessed information together and reducing the need for joins.
- Stores related data as sub-documents in one document.
- Reduces the need for cross-collection joins.
- Improves read performance for related data.
- Supports atomic updates on a single document.

# Embedded Data Model example

- If we obtain student information in three different documents, Personal\_details, Contact, and Address, we can embed all three in a single one, as shown below.

```
{
  _id: ,
  Std_ID: "987STD001"
  Personal_details:{
    First_Name: "Ron",
    Last_Name: "Smith",
    Date_Of_Birth: "1999-08-26"
  },
  Contact: {
    e-mail: "ron_smith.123@gmail.com",
    phone: "9987645673"
  },
  Address: {
    city: "Karnataka",
    Area: "BTM2ndStage",
    State: "Bengaluru"
  }
}
```

# Embedded Data Model Advs and Disadv

## Advantages of the Embedded Model:

- **Faster reads:** All related data is fetched together, reducing the need for multiple queries.
- **Atomic operations:** Data is stored in a single document, ensuring atomic write operations (a single document is updated at once).

## Disadvantages:

- **Data redundancy:** Data may be duplicated, increasing storage usage.
- **Limited by document size:** MongoDB documents have a size limit of 16MB.

# Normalized Data Model

- The Normalized Data Model stores related data in separate collections and links them using references, reducing duplication and supporting complex relationships across collections.
- Links related data across collections using references (IDs).
- Reduces duplication and supports complex relationships.
- Suitable for large, shared, hierarchical datasets.

# Normalized Data Model Example

- Created multiple collections for storing students data which are linked with `_id`.

## Student:

```
{
  _id: <StudentId101>,
  Std_ID: "10025AE336"
}
```

## Personal\_Details:

```
{
  _id: <StudentId102>,
  stdDocID: " StudentId101",
  First_Name: "Ron",
  Last_Name: "Smith",
  Date_Of_Birth: "1999-08-26"
}
```

## Contact:

```
{
  _id: <StudentId103>,
  stdDocID: " StudentId101",
  e-mail: "ron_smith.123@gmail.com",
  phone: "9987645673"
}
```

## Address:

```
{
  _id: <StudentId104>,
  stdDocID: " StudentId101",
  city: "Karnataka",
  Area: "BTM2ndStage",
  State: "Bengaluru"
}
```



# Normalized Data Model Advs & Disadv

## Advantages of the Normalized Model:

- **Reduces data redundancy:** Common data (e.g., personal details) is stored once and referenced where necessary.
- **Efficient for many-to-many relationships:** Works well for complex relationships between data.

## Disadvantages:

- **Slower reads:** Requires multiple queries or joins to retrieve related data, which can impact performance.
- **More complex to manage:** Managing relationships between collections can be more difficult.

# Advantages Of Data Modeling in MongoDB

Data modeling in MongoDB helps design efficient schemas that improve performance, simplify development, and support long-term scalability and maintenance.

- Boosts application performance with optimized schema design.
- Speeds up development cycles through easier object mapping.
- Improves data understanding, standards, and validation.
- Supports future scalability and maintainability of applications.

# Best Practices in Data Modelling

Some important points to consider while creating a data model for MongoDB database are:

- **Design Based on Application Needs:** Model the data according to how it will be accessed (e.g., frequent queries, updates, etc.).
- **Separate Collections for Different Entities:** Store unrelated data in separate collections to reduce complexity.
- **Embed Frequently Accessed Data:** For better performance, embed related data that will be accessed together in a single document.
- **Avoid Complex Joins:** Perform joins during writes, not reads, to optimize query performance.
- **Use Aggregation:** Leverage MongoDB's aggregation framework to perform complex queries and transformations.